SIVF: GPU-Resident IVF Index for Streaming Vector Search

Anonymous Author(s)

Abstract

GPU-accelerated Inverted File (IVF) index is one of industry standards for large-scale vector search but rely on static memory layouts that hinder real-time mutability. Our benchmark and analysis reveal that existing designs necessitate expensive CPU-GPU data transfers for index updates, causing system latency to spike from milliseconds to seconds in streaming scenarios. We propose SIVF (Streaming Inverted File), a GPU-native architecture that enables high-velocity, in-place mutation via slab-based memory allocation and lock-free validity bitmaps. Evaluation against state-of-the-art vector search baselines (GPU IVF, GPU Flat, HNSW, NSG, LSH) with multiple real-world datasets (SIFT, T2I, GIST) demonstrates that SIVF reduces deletion latency to sub-millisecond levels, outperforming baselines by orders of magnitude. Moreover, SIVF achieves up to 105× higher ingestion throughput and 266× end-to-end sliding window speedup with negligible memory overhead (<0.8%).

ACM Reference Format:
Anonymous Author(s). 2026. SIVF: GPU-Resident IVF Index for Streaming Vector Search. In . ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Real-world vector search applications, ranging from real-time recommendation engines to fraud detection systems, increasingly operate on streaming data where timeliness is critical. These systems necessitate a sliding window model where expired vectors must be invalidated promptly as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads. To quantify the gap between insertion and eviction performance, we conducted benchmarks using the industry-standard Faiss library [20] on an NVIDIA RTX 6000 GPU [23]. We utilized the SIFT1M dataset [22] (128-dimensional) to simulate a realistic sliding window scenario. Specifically, we initialized a standard `IVF1024` index pre-populated with 100,000 vectors and measured the wall-clock latency of ingesting a batch of 10,000 new vectors and subsequently evicting the oldest 10,000 IDs.

As illustrated in Figure 1, we observe a drastic performance asymmetry. While GPU parallelism accelerates the insertion of the 10K-vector batch to approximately 28.6 ms, the eviction process for the same batch size is significantly slower, spiking the latency to

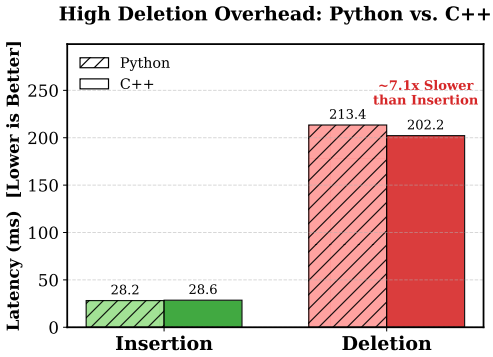

Figure 1: The asymmetry between insertion and deletion latency on GPU indices.

202.2 ms. This high latency is observed in the native C++ implementation, which performs nearly identically to the Python wrapper (213.4 ms). The negligible difference between the compiled C++ core and the interpreted Python binding suggests that the bottleneck is not an implementation detail: If the issue were merely inefficient code, the C++ implementation would be expected to significantly outperform Python. The lack of such divergence indicates that both are bound by the same architectural limitation: the absence of a GPU-resident deletion operator, which forces expensive CPU-GPU roundtrips for every eviction operation.

Our analysis of the Faiss source code [6] reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing. In the library's architecture, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support. Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. That is, to delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing GPU compute throughput, capping the maximum sustainable ingestion rate in streaming scenarios.

The absence of a GPU-native eviction operator is not an oversight but a consequence of the complexity involved in managing dynamic data structures within VRAM. Unlike CPU indices that utilize flexible dynamic arrays, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory coalescing and bandwidth utilization. Implementing in-place deletion on the GPU presents three orthogonal challenges. First, maintaining contiguous arrays requires dynamic memory management. Frequent reallocation and compaction of inverted lists within VRAM lead to severe

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

memory fragmentation and costly data movement. Second, concurrent modification requires complex synchronization. Managing fine-grained locks to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures introduces significant latency. Third, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints render naive in-place GPU eviction prohibitively inefficient.

To resolve these architectural limitations, we propose SIVF, i.e., Streaming Inverted File, a GPU indexing architecture designed to support high-throughput insertion and deletion without relying on host interaction. SIVF introduces a triad of mechanisms to enable efficient in-place mutability on GPUs. We first address the memory management overhead by abandoning the contiguous memory layout in favor of a *slab allocation* mechanism. GPU memory is pre-allocated as a pool of fixed-size blocks, and inverted lists are implemented as chained blocks. When a list grows, a new block is atomically linked from the free pool, virtually eliminating memory fragmentation and avoiding the cost of data compaction. To solve the synchronization challenge, we decouple logical deletion from physical removal using a bitmap-based lazy eviction strategy. Each memory block is associated with a validity bitmap; eviction is reduced to a single atomic bit-flip, allowing thousands of concurrent threads to operate without contention. Finally, to eliminate the $O(N)$ scan overhead, SIVF maintains a compact, GPU-resident address translation table (ATT). This table maps active vector IDs to their physical memory coordinates, enabling $O(1)$ location resolution during deletion.

We have implemented SIVF within the Faiss framework, integrating it as a new index class `GpuIndexSIVF`. Our implementation leverages CUDA C++ for device-side kernels and employs a dual-view memory management strategy to maintain consistency between host and device states. The slab allocator, bitmap eviction, and address table are encapsulated within a `SlabManager` class that abstracts the complexity of dynamic memory operations on the GPU. We have evaluated SIVF against a comprehensive set of baselines, including GPU Flat, GPU IVF, HNSW, NSG, and LSH indices, using multiple real-world datasets such as SIFT1M, T2I, and GIST. Our experiments demonstrate that SIVF achieves up to $13,300\times$ reduction in deletion latency (e.g., from 11.8 seconds to 0.89 ms on GIST) and improves ingestion throughput by $36\times$ to $105\times$ compared to existing GPU indices. In end-to-end sliding window scenarios, SIVF delivers a $161\times$ to $266\times$ speedup with negligible memory overhead (<0.8%), effectively closing the gap between static and streaming vector search performance.

In summary, this paper makes three technical contributions:

- We identify the CPU-GPU bottleneck in existing GPU vector indices, demonstrating that the lack of native deletion support renders them unsuitable for streaming vectors.
- We propose a GPU-native index, namely SIVF, that synthesizes slab-based memory management, validity bitmaps, and on-device address translation. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead without costly CPU-GPU synchronization.
- We implement SIVF in the state-of-the-art vector search library Faiss and evaluate SIVF against a broad spectrum of baselines (GPU Flat, GPU IVF, HNSW, NSG, and LSH) on a variety of real-world datasets, such as SIFT, T2I, and GIST.

Paper Organization. The remainder of this paper is organized as follows. Section 2 reviews the existing landscape of ANN search, distinguishing between CPU-based dynamic methods and static GPU-accelerated indices. Section 3 details the architectural design of SIVF, elaborating on the slab-based memory allocator, the validity bitmap mechanism, and the GPU-resident address translation table. Section 4 presents our experimental methodology and a comprehensive evaluation of SIVF against state-of-the-art baselines. Finally, Section 5 concludes the paper and outlines directions for future research.

2 Related Work

Optimization of Graph-Based Indices. Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, frameworks like VSAG [50] optimize memory access patterns and parameter tuning, while SOAR [40] improves indexing structures. Of note, CAGRA [35] establishes a new state-of-the-art for static graph construction and search throughput on GPUs; however, it is fundamentally designed for write-once-read-many workloads and lacks native support for the efficient, fine-grained in-place deletions required by streaming applications. Navigational efficiency is a key optimization target; SHG [11] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [31] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [21] utilizes angular distance guidance to filter irrelevant neighbors, DADE [5] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [38] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids. Efforts to improve graph construction and maintenance include LIGS [3], which employs locality-sensitive hashing (LSH) to simulate proximity graphs for faster updates, and CSPG [47], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [13] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [42] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [45] provide guarantees on result quality using clustering-based indexing.

Attribute-Filtered and Hybrid Search. The integration of structured constraints with vector search has led to a surge in Filtered ANN research. Li et al. [26] provide a comprehensive taxonomy and benchmark of these methods. A primary focus is handling dynamic updates and specific filter types such as ranges or windows. For range filtering, UNIFY [29] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [19], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [49], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [36] propose dynamic segment graphs to handle mixed streams of data and queries. Regarding specific constraints, Wang et

al. [43] present a robust framework for general attribute constraints. Engels et al. [7] focus on window filters, while WoW [44] develops a window-graph based index to handle window-to-window incremental construction.

Quantization and Scalability. To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [10] and its extended version [9] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [12] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [18] utilizes low-rank matrix factorization for compression. In terms of system architecture, Tagore [28] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [46] employs a multi-granularity partition strategy to balance load. WebANNS [30] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [17] analyze the worst-case performance limits of popular implementations. Finally, DARTH [1] introduces declarative recall targets through adaptive early termination to balance performance and result quality.

GPU Systems and Optimizations. Recent HPC work highlights how careful kernel and data-layout design can unlock GPU performance across data-intensive workloads. FZ-GPU demonstrates a fully parallel compression pipeline with both warp-aware bitwise operations and shared-memory efficiency [48]. For sparse linear algebra, SpMV designs reduce preprocessing overhead while improving load balance and memory access locality [2], and GPU-aware preconditioning further emphasizes locality and coalescence as first-order concerns on modern GPU architectures [24]. Beyond compute kernels, GPU-enabled asynchronous checkpoint caching and prefetching treat GPU memory as a first-class tier to improve end-to-end throughput for I/O-heavy workflows [33]. Format choices and composition are also critical for irregular workloads, as shown by automatic sparse format composition for SpMM that avoids expensive autotuning while delivering strong performance [37]. Another line of work focuses on reliability, debugging, and cluster-scale GPU management, which together shape how GPU-centric systems are built and operated. GPU-FPX provides low-overhead floating-point exception tracking for NVIDIA GPUs [27], FPBOXer improves scalable input generation for triggering floating-point exceptions in GPU programs [41], and FloatGuard extends whole-program exception detection to AMD GPUs and highlights cross-vendor portability concerns [34]. At the application and cluster level, SIMCoV-GPU studies multi-node, multi-GPU acceleration for irregular agent-based simulations [25], FASOP automates fast search for near-optimal transformer parallelization on heterogeneous GPU clusters [16], and serverless platforms motivate GPU sharing and scheduling mechanisms such as ESG and FluidFaaS [14, 15]. These works reinforce the importance of aligning work granularity with warps, minimizing synchronization overhead, and designing memory layouts that preserve locality, which are relevant to GPU-resident indexing under dynamic updates.

3 Design and Implementation of SIVF

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design exhibits severe limitations in streaming scenarios. First, resizing an inverted list requires allocating a larger memory region and copying existing data, introducing synchronization stalls and high latency. Second, frequent insertions and deletions of variable-length lists exacerbate memory fragmentation and reduce allocator efficiency on GPUs.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a contiguous GPU memory region and manages it as a pool of fixed-size blocks, referred to as *slabs*. Inverted lists are therefore represented as linked chains of slabs. This design transforms dynamic list growth into constant-time block allocation while enabling efficient in-place mutation through lightweight metadata updates.

3.1.1 Slab Layout and Metadata. A *slab* is defined as the fundamental unit of storage. Each slab has a fixed capacity C, and we set $C = 32$ to align with the NVIDIA warp size, thereby enabling coalesced memory access when threads within a warp operate on the same slab. A Slab S_i consists of two regions:

- *Data Payload*: Stores vectors contiguously. For a vector space of dimension D, the payload size is
$$L_p = C \times D \times \text{sizeof(float)}.$$
- *Metadata Header*: Stores control information required for traversal and mutation. The metadata of slab S_i is represented as
$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle, \quad (1)$$
where n_{next} denotes the identifier of the next slab in the logical list, b_{valid} is a 32-bit validity bitmap indicating occupied slots, and c_{valid} records the number of valid vectors stored in the slab.

Logical deletion is implemented by atomically clearing the corresponding bit in b_{valid}, yielding constant-time removal without triggering memory compaction or data movement to CPUs.

3.1.2 Conflict-Free Slab Allocation. General-purpose device-side memory allocators incur substantial synchronization and fragmentation overhead when invoked from GPU kernels. SDMA instead manages slabs from a pre-allocated pool. On initialization, a device kernel constructs a free-list array and sets a global stack pointer P_{top} to the pool size. Allocation obtains a fresh slab identifier by atomically decrementing P_{top}:

$$s_{id} = \texttt{atomicSub}(P_{top}, 1) - 1. \quad (2)$$

In lock-free allocators, reusing memory identifiers may lead to the "ABA" problem, where a shared variable changes from value A to B and back to A between two observations, causing concurrent threads to falsely assume that the system state remains unchanged. In GPU memory management, this may occur if a slab identifier is reclaimed and reassigned while another thread still holds a stale reference. SDMA avoids this issue by ensuring that slab identifiers are never reused while concurrent kernels may still observe them. Deletion only invalidates logical occupancy via the bitmap and

does not immediately recycle slab identifiers back to the pool. This design keeps the meaning of an allocated slab identifier stable during execution and avoids additional versioning mechanisms.

3.1.3 Virtual Addressing via Address Table. To decouple logical vector identifiers from physical storage locations, SDMA introduces a global Address Table that implements a translation function

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle, \tag{3}$$

where s_{id} denotes the slab identifier and o_{slot} denotes the slot offset within that slab, with $0 \leq o_{slot} < C$.

The address table is initialized with an invalid sentinel value for all entries. Upon insertion, the table is updated to record the physical location of each vector. During deletion, the system queries $\mathcal{T}(v_{id})$ to directly locate the corresponding slab and slot, enabling constant-time invalidation without scanning inverted lists.

3.1.4 System Implementation. We implemented SDMA in CUDA C++ within the Faiss framework using a dual-view architecture. On the host side, GPU memory resources are managed using RAII-compliant containers encapsulated in a `SlabManager` class, including slab metadata, slab data, address table, and allocator state. These resources are projected into a lightweight `SlabManagerDevice` structure containing raw pointers and scalar parameters passed by value to device kernels.

Index integration is implemented in `GpuIndexSIVF`. During initialization, the index selects a safe slab pool size and derives a consistent maximum vector capacity to prevent device memory overflow when slabs are appended. The index maintains an array of per-list head pointers (`list_heads`) initialized to an invalid value. Insertions first perform coarse quantization using the existing Faiss GPU quantizer to obtain list assignments, and then invoke a GPU append kernel that writes vectors into the slab chains. Deletions use the address table to locate and invalidate target slots in place.

Algorithm 1 summarizes the allocator and deletion logic. Insertion resolves the target inverted list, attempts to reserve a free slot in the current head slab, and allocates and links a new slab if needed. The address table is updated after the physical location is finalized. Deletion performs a direct address table lookup followed by an atomic bit clear, leaving slab storage intact.

Algorithm 1 Core Operations of SDMA

Global State: slab pool metadata and data, per-list head pointers $H[\cdot]$, stack pointer P_{top}, address table $\mathcal{T}$
Input: vector identifier v_{id}, vector x, list assignment ℓ
procedure INSERT(v_{id}, x, ℓ)
  $s \leftarrow H[\ell]$
  while s is invalid **or** no free slot exists in slab s **do**
    $s_{new} \leftarrow$ `atomicSub`$(P_{top}, 1) - 1$
    Initialize metadata s_{new} with $b_{valid} = 0$, $n_{next} = s$
    Update $H[\ell]$ to s_{new} if unchanged, otherwise retry
    $s \leftarrow H[\ell]$
  end while
  Reserve a free slot o in slab s via atomic update of c_{valid}
  Write x to slab data$[s][o]$
  Atomically set bit o in b_{valid}
  $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$
end procedure
procedure DELETE(v_{id})
  $\langle s, o \rangle \leftarrow \mathcal{T}(v_{id})$
  if $\langle s, o \rangle$ is valid **then**
    Atomically clear bit o in b_{valid} of slab s
    Mark $\mathcal{T}(v_{id})$ as invalid
  end if
end procedure

3.2 Lock-free Parallel Ingestion

Many GPU-based ANN indices are optimized for static or bulk ingestion and commonly rely on sorting, segmented scans, or bulk-copying to build inverted lists. While these approaches can achieve high throughput, they often incur high end-to-end latency for streaming updates. SIVF instead implements a fully parallel insertion kernel in which each CUDA thread ingests one vector and performs lock-free appends to inverted lists. Concurrency is handled using atomic slot reservation in the current head slab and speculative slab expansion under contention. Correctness under CUDA's relaxed memory consistency is ensured by device-wide memory fences and a publish protocol based on a validity bitmap.

At a high level, each SIVF list is represented as a singly linked chain of fixed-capacity slabs. Each slab stores up to $C = 32$ vectors and maintains a counter `valid_count`, a 32-bit `validity_bitmap`, and a link field `next_slab_idx`. Insertion follows a two-stage protocol. First, a thread attempts to reserve a slot in the current head slab. If the head slab is absent or full, the thread allocates a new slab from the global pool and attempts to publish it as the new head using compare-and-swap (CAS). In both cases, the inserted vector becomes visible to concurrent readers only when the corresponding validity bit is set after a memory fence.

3.2.1 Atomic Slot Reservation. Ingestion begins with coarse quantization, assigning each vector to a list ℓ. A thread reads the current head slab index $h = H[\ell]$. If $h \neq -1$, the thread attempts to reserve a writing slot by conditionally incrementing the head slab counter. Let c be the value of `valid_count`. If $c < C$, the thread executes:

$$\text{success} = \texttt{atomicCAS}(\&\text{slab}[h].\text{valid\_count}, c, c+1) == c. \tag{4}$$

When the above check succeeds, the reserved slot index is $o = c$. This conditional CAS-based increment ensures that exactly one thread claims each slot index and avoids counter overflow under contention. Threads that fail the CAS retry by re-reading the head and counter.

After reserving a slot, the thread writes the vector payload into the slab data region, writes the user identifier into a parallel identifier buffer indexed by `(slab, slot)`, and updates the address table entry for the user identifier to a compact coordinate encoding $\langle s_{id}, o_{slot} \rangle$. Only after these writes are completed does the thread execute `__threadfence()` and set the corresponding bit in `validity_bitmap` via `atomicOr`. The validity bit is the publication signal for readers, so this ordering ensures that a concurrent search that observes an enabled bit never reads uninitialized payload or a missing address translation.

3.2.2 Speculative Slab Expansion. If the head slab is absent ($h = -1$) or full, the thread expands the list by allocating a new slab from the global pool. Allocation decrements a global stack pointer

free_list_top and retrieves a slab index from free_list. If the pool is exhausted, the thread restores the pointer and returns.

After allocation, the thread initializes the new slab metadata by setting valid_count to 1, clearing validity_bitmap to 0, and setting next_slab_idx to the previous head h. A __threadfence() is executed before attempting to publish the new slab as the list head using CAS:

$$\text{success} = \texttt{atomicCAS}(\&H[\ell], h, s_{new}) == h. \tag{5}$$

Equation 5 defines the linearization point of list expansion. Only one contending thread succeeds in updating the head pointer, and its newly allocated slab becomes immediately visible to concurrent writers and readers that traverse from the head.

If the head publication CAS fails due to contention, the allocated slab is not returned to the free list. Instead, the thread retries the insertion loop. This leak-on-failure policy avoids additional synchronization on allocator state under contention and prevents hazards that may arise from reclaiming slab identifiers while other threads may still observe stale references. In practice, the overhead is controlled by provisioning sufficient slab pool capacity so that occasional contention-induced leakage does not affect ingestion.

3.2.3 Kernel-Level Memory Ordering and Publication. SIVF enforces safe publication under CUDA's weak memory consistency using __threadfence() at two critical points. First, before a thread sets a slot's validity bit, it fences to ensure that payload writes, identifier buffer writes, and address table updates become globally visible. Second, before a thread publishes a newly initialized slab as the list head, it fences to ensure that slab metadata is visible before any concurrent traversal observes the head update. These fences prevent readers from observing a published structural pointer or validity bit that refers to partially initialized state.

3.2.4 Micro-architectural Considerations.

Register-efficient addressing. SIVF represents slab links using 32-bit slab indices within a pre-allocated region rather than 64-bit device pointers. This reduces register pressure and simplifies address computation in the critical path. The address table stores a compact 64-bit coordinate encoding the slab index and slot offset, enabling constant-time deletion without list scans.

Contention behavior and forward progress. Insertion experiences high contention when many vectors map to the same centroid. Most insertions complete through slot reservation when the head slab has available capacity. When the head becomes full, contention shifts to head publication, after which subsequent threads quickly observe the new head and resume slot reservation. The kernel bounds retries with a fixed attempt limit.

Intra-slab coalescing. Vectors within each slab are stored contiguously. When multiple threads append to adjacent slots, payload writes to slab data and identifier buffer writes are naturally coalesced, preserving memory bandwidth under streaming ingestion.

Algorithm 2 summarizes the per-thread ingestion protocol implemented by the insertion kernel. The protocol first attempts slot reservation on the current head slab using CAS on valid_count. If successful, it writes payload, identifier, and address table entry, then publishes the slot by setting the validity bit after a device-wide fence. If the head is absent or full, it allocates and initializes a new slab and attempts to publish it as the new head using CAS on the head pointer, again with a fence before publication. If head publication fails, it retries without reclaiming the allocated slab.

Algorithm 2 Lock-free ingestion protocol in SIVF

```
Input: vector x, identifier v_id, list assignment ℓ
Global: head array H[·], slab metadata, slab data, free list, stack pointer P_top, Address Table T
attempts ← 0
while attempts < 1000 do
    attempts ← attempts + 1
    h ← H[ℓ]
    if h ≠ −1 then
        c ← slab_meta[h].valid_count
        if c < C then
            if atomicCAS(&slab[h].valid_cnt) == c then
                Write payload x to slab data[h][c]
                Write identifier to slab id buffer[h][c]
                T(v_id) ← ⟨h, c⟩
                __threadfence()
                atomicOr(&slab_meta[h].validity_bitmap)
                return
            end if
        end if
    end if
    t ← atomicSub(P_top, 1)
    if t ≤ 0 then
        atomicAdd(P_top, 1)
        return
    end if
    s_new ← free_list[t − 1]
    Set slab_meta[s_new].valid_count ← 1
    Set slab_meta[s_new].validity_bitmap ← 0
    Set slab_meta[s_new].next ← h
    __threadfence()
    if atomicCAS(&H[ℓ], h, s_new) == h then
        Write payload x to slab data[s_new][0]
        Write identifier to slab id buffer[s_new][0]
        T(v_id) ← ⟨s_new, 0⟩
        __threadfence()
        atomicOr(&slab_meta[s_new].validity_bitmap)
        return
    end if
end while
```

3.3 Coalesced Search on Non-Contiguous Slabs

SIVF replaces contiguous inverted lists with linked slabs, which introduces pointer chasing during traversal. To retain high throughput on GPUs, we design a warp-cooperative search kernel that maps the slab capacity to the warp width. The kernel is optimized for the execution model in which one warp collaboratively processes one query and evaluates one slab per traversal step.

3.3.1 Warp-Cooperative Slab Traversal. SIVF launches the search kernel with one thread block per query and 32 threads per block. This design assigns each query to a single warp. For a query vector $\mathbf{q} \in \mathbb{R}^D$, the warp first stages the query into shared memory to reduce redundant global loads:

$$\mathbf{q} \rightarrow \texttt{shared\_query}.$$

The kernel then probes *nprobe* coarse lists. For each probed list, the warp traverses the slab chain starting from the list head. At each slab S, thread t_j is responsible for slot j in that slab. The thread consults the slab validity bitmap and only evaluates the distance if the corresponding bit is set:

$$\text{valid}(S, j) \Leftrightarrow ((b_{valid} \gg j)\ \&\ 1) == 1. \tag{6}$$

If valid, the thread loads the vector stored at (S, j) and computes the squared L_2 distance:

$$d(\mathbf{q}, \mathbf{x}_{S,j}) = \sum_{k=1}^{D} (q_k - x_{S,j,k})^2. \tag{7}$$

The slab pointer is advanced using the metadata `next_slab_idx`. The kernel also includes safety checks that bound the traversal length and break on degenerate self-loops, preventing potential non-terminating traversal under unexpected corruption.

3.3.2 Top-k Maintenance and Warp Aggregation. Maintaining top-k results without global contention is achieved using a two-stage aggregation scheme. Each thread maintains a small top-k list in registers. Upon computing a candidate distance, the thread inserts it into its local top-k list using an in-register insertion routine. This avoids shared data structures and eliminates inter-thread contention during candidate generation.

After all probed lists are traversed, the warp aggregates results. Each thread writes its local top-k list to shared memory. A single thread then performs a deterministic merge across the 32 per-thread lists and outputs the final top-k for the query. This design trades a small amount of serial work for a contention-free aggregation path and avoids additional warp-level primitives.

3.3.3 Correctness Under Concurrent Ingestion. SIVF supports concurrent ingestion by using a publish protocol based on the validity bitmap. The ingestion kernel writes the payload, updates the identifier buffer and address table entry, and then executes `__threadfence()` before setting the corresponding validity bit via an atomic operation. The search kernel consults the validity bit as the sole signal that a slot is present. Therefore, if search observes a valid bit, the payload and identifier for that slot have been made globally visible prior to publication. This ensures that search does not read partially initialized vectors from concurrent insertions.

Algorithm 3 describes the warp-cooperative search procedure of SIVF. We launch one CUDA block per query with 32 threads, so each query is processed by a single warp. At the beginning of the kernel, the warp cooperatively stages the query vector into shared memory to avoid redundant global loads during list traversal. For each query, we probe *nprobe* coarse lists returned by the quantizer. For a probed list, the warp traverses its slab chain starting from the list head pointer. At each slab, thread t_j corresponds to slot j and consults the slab validity bitmap. If the j-th bit is set, the thread loads the vector payload from the slab data region, computes the squared L_2 distance to the shared query, retrieves the corresponding user identifier from the slab identifier buffer, and updates its local top-k candidates. Traversal proceeds by following the `next_slab_idx` pointer recorded in the slab metadata. To ensure termination under unexpected corruption, we additionally bound the traversal length and break on self-loops.

Algorithm 3 Warp-cooperative search in SIVF

```
Input: queries Q, probed list ids coarse_ids, head array H[·]
Input: slab metadata and data, buffer slab_ids, k, nprobe
Output: top-k distances and labels for each query
for each query index q in parallel do
    Launch one warp for query q
    Copy Q[q] into shared memory
    Initialize per-thread local top-k lists in registers
    for p = 0 to nprobe − 1 do
        ℓ ← coarse_ids[q, p]
        if ℓ is invalid then
            continue
        end if
        s ← H[ℓ]
        while s is valid and traversal bound not exceeded do
            md ← slab_meta[s]
            if md.next = s then
                break
            end if
            if j < 32 and md.validity_bitmap[j] is set then
                Load vector x_{s,j} from slab data
                Compute d(q, x_{s,j})
                id ← slab_ids[s, j]
                Insert (d, id) into local top-k
            end if
            s ← md.next
        end while
    end for
    Write local top-k lists to shared memory
    One thread merges 32 lists and writes final top-k outputs
end for
```

3.4 Bitmap-based Lazy Eviction

Traditional vector deletion in IVF indices often requires locating the target element inside an inverted list and physically compacting memory to close the gap, which can incur a cost proportional to the list length and trigger substantial global memory traffic. This overhead fundamentally limits deletion throughput in streaming workloads. SIVF adopts bitmap-based lazy eviction, which decouples logical validity from physical residency. Deletion is reduced to a constant-time metadata update by combining a global Address Translation Table (ATT) with atomic bitwise invalidation of per-slab validity bitmaps.

3.4.1 Address Translation Table (ATT). SIVF maintains a global table $\mathcal{T}$ that maps a user identifier u to its physical coordinate in the slab pool. The table is implemented as a flat array of 64-bit entries. For an identifier u, $\mathcal{T}[u]$ encodes the slab index and slot

offset as:

$$\mathcal{T}[u] = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}, \tag{8}$$

where $\text{idx}_{slot} \in [0, 31]$. The table is initialized with an invalid sentinel value, denoted as INVALID, for all entries. This design enables direct random access to the target slot without traversing inverted lists.

3.4.2 Atomic Bitwise Invalidation. Given $\mathcal{T}[u]$, the deletion kernel resolves $(\text{idx}_{slab}, \text{idx}_{slot})$ and invalidates the corresponding slot via the slab validity bitmap. Let $\mathcal{B}_i$ denote the 32-bit bitmap of slab S_i. The deletion operation clears the target bit using an atomic read-modify-write:

$$\mathcal{B}_i \leftarrow \texttt{atomicAnd}(\mathcal{B}_i, \sim (1 \ll \text{idx}_{slot})). \tag{9}$$

The vector payload is not moved or overwritten. Search kernels consult the bitmap when scanning a slab and ignore slots whose bits are cleared, so deletion immediately removes the vector from the search space without compaction.

3.4.3 Idempotence and Consistency. Deletion requests may include duplicates or may race with other deletions. SIVF treats deletion as an idempotent operation. After the atomic bitmap update, the kernel checks whether the bit was previously set by inspecting the pre-update value returned by `atomicAnd`. Only if the bit transitioned from 1 to 0 does the kernel decrement the slab counter `valid_count` and increment a global deletion counter. Finally, the kernel marks $\mathcal{T}[u] \leftarrow$ INVALID to prevent future deletion requests from dereferencing a stale coordinate.

This protocol provides a clear correctness contract with concurrent search and insertion. Search considers a slot present only if its validity bit is set. Insertion sets the validity bit only after payload and identifier writes are completed and a device-wide memory fence is executed. Deletion clears the validity bit via an atomic operation, so a deleted slot is excluded from search as soon as the bit is cleared. The ATT invalidation is a secondary safety step that prevents repeated deletions from accessing recycled or stale coordinates.

Deletion performs a constant number of random memory accesses: one read of $\mathcal{T}[u]$, one atomic update of a 32-bit bitmap, and one write of the 64-bit ATT entry to the invalid sentinel. The data payload remains untouched, avoiding bandwidth-heavy compaction. In addition, the implementation optionally updates a 32-bit counter in the slab metadata and a global 32-bit deletion counter for bookkeeping. Overall, the dominant deletion work is independent of the inverted list length, explaining the large speedups observed in our evaluation.

Algorithm 4 presents the bitmap-based lazy eviction procedure implemented by the SIVF deletion kernel. Each GPU thread processes one identifier u in the deletion batch. The thread first performs a constant-time lookup in the ATT to obtain the encoded coordinate $\mathcal{T}[u]$. If the entry equals the invalid sentinel, the identifier is already absent and the thread terminates without side effects. Otherwise, the thread decodes the slab index and slot offset from the 64-bit coordinate, locates the corresponding slab metadata, and clears the target slot by applying an atomic bitwise operation to the slab validity bitmap. The atomic operation returns the previous bitmap value, which allows the kernel to determine whether the bit was previously set. Only when the bit transitions from 1 to 0 does the kernel treat the deletion as successful and perform two bookkeeping updates: it atomically decrements the slab `valid_count` and atomically increments a global `deleted_count`. Finally, the kernel writes the invalid sentinel back to $\mathcal{T}[u]$ to prevent future deletion requests from dereferencing a stale coordinate.

Algorithm 4 Bitmap-based lazy eviction in SIVF

```
Input: deletion identifiers U = {u_1, ..., u_m}
Global: Address Table T, slab metadata with bitmap B and counter valid_count
for each u in U in parallel do
    coord ← T[u]
    if coord = INVALID then
        continue
    end if
    idx_slab ← coord ≫ 32
    idx_slot ← coord & (2^32 − 1)
    mask ←  ∼ (1 ≪ idx_slot)
    old ← atomicAnd(B_{idx_slab}, mask)
    if ((old ≫ idx_slot) & 1) = 1 then
        atomicSub(&valid_count, 1)
        atomicAdd(&deleted_count, 1)
        T[u] ← INVALID
    end if
end for
```

4 Evaluation

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [6], integrating our proposed slab-based memory management and lock-free update mechanisms into its core indexing architecture. Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads. We have open-sourced our complete implementation and evaluation scripts on GitHub.

Implementing SIVF required substantial modifications to the Faiss GPU backend and associated experimental infrastructure. In total, the implementation involves approximately 9,000 lines of code changes across more than 45 source files. Core system changes are concentrated in the GPU index implementation, incorporating new GPU-native insertion, deletion, and search operators (e.g., `GpuIndexSIVF`, `SIVFAppend`, `SIVFSearch`), alongside a custom slab-based memory manager (`SlabManager`) and lock-free metadata structures. Furthermore, we developed a comprehensive benchmarking suite that extends beyond standard metric testing to include end-to-end streaming simulations, memory scalability analysis, and automated visualization scripts, ensuring a rigorous and reproducible evaluation.

4.1 Experimental Setup

To ensure a comprehensive evaluation, we adopt a two-tiered baseline strategy. First, our primary baseline is the standard Faiss GPU IVF, which serves as a direct architectural counterpart; this allows us to isolate the performance gains specifically attributable

to our proposed memory management innovations, excluding confounding factors from differing algorithmic paradigms. Second, to position SIVF within the broader indexing landscape, we compare against representative non-IVF indices, including GPU Flat (brute-force), HNSW [32] (dynamic graph), NSG [8] (static graph), and LSH [4] (hashing).

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [23] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM. For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 memory. The system runs on a Ubuntu 24.04 environment with the CUDA toolkit installed.

To ensure statistical reliability, each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity. Unless explicitly stated (as in Section 4.5), our experiments utilize synthetic datasets consisting of random vectors generated from a uniform distribution. We employ this configuration as a neutral baseline to isolate system-level overheads (e.g., memory allocation, PCIe transfer) from data distribution effects. Note that because uniform data lacks the clustering structure inherent in real-world applications, the performance advantages of SIVF in microbenchmarks are primarily architectural and appear more conservative than those observed on real-world datasets.

4.2 Microbenchmark: Ingestion

We evaluate ingestion throughput against the standard Faiss GPU IVFFlat baseline, varying database size (N_B: 1M–4M) and cluster count (n_{list}: 1024–16384). As shown in Figure 2, SIVF achieves consistent performance advantages with up to 2.88× speedup.

Scalability (Fig. 2a). SIVF maintains a stable throughput of ≈4.2M vec/s as N_B increases, significantly outperforming the baseline (≈1.7M vec/s). This stability validates our lock-free SlabManager, which ensures $O(1)$ insertion cost independent of index occupancy, thereby avoiding the contiguous memory resizing overheads inherent to the baseline.

Granularity Impact (Fig. 2b). Performance correlates with clustering granularity. At low $n_{list} = 1024$, localized memory writes allow SIVF to peak at ≈6.1M vec/s (2.88× speedup). At high $n_{list} = 16384$, scattered write patterns reduce throughput for both systems; however, SIVF's linked-slab architecture proves more resilient than the baseline, maintaining a 2.0×–2.4× advantage even under high fragmentation.

Overall Speedup (Fig. 2c). SIVF outperforms the baseline across all configurations, with speedups typically ranging from 2.4× to 2.9×. Even under maximum contention (4M vectors, $n_{list} = 1024$), SIVF retains a 1.79× advantage, confirming the efficiency of the non-blocking, slab-based pipeline for streaming scenarios.

4.3 Microbenchmark: Search

We evaluate query performance against the Vanilla Faiss GPU baseline. While SIVF trades memory contiguity for mutability, Figure 3 confirms competitive efficiency, reaching up to 91% of baseline performance in optimized configurations.

Scalability (Fig. 3a). SIVF throughput scales predictably from ≈383k to 118k QPS as database size grows (100k–500k, $n_{list} = 4096$). The trend closely mirrors the baseline, validating the stability of our slab traversal logic under increasing load.

Granularity Impact (Fig. 3b). Performance improves with finer clustering. Increasing n_{list} from 1024 to 4096 boosts SIVF throughput from 70k to 118k QPS (500k vectors). This reduction in per-probe distance computations ensures robust throughput across settings.

Query Efficiency (Fig. 3c). The efficiency ratio, which is defined as ($QPS_{SIVF}/QPS_{Vanilla}$), highlights the effectiveness of warp-level optimizations. In sparse scenarios ($n_{list} = 16384$, 100k vectors), SIVF achieves 0.91× efficiency, nearly matching contiguous arrays. As lists lengthen (500k vectors), the ratio stabilizes between 0.40× and 0.52×, reflecting the necessary physical trade-off of pointer chasing for dynamic memory support.

4.4 Microbenchmark: Deletion

We evaluate the efficiency of the vector deletion mechanism in SIVF by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of one million 128-dimensional vectors.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.20 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from 4.9×10^4 vec/s in the baseline to nearly 1.47×10^7 vec/s in SIVF.

4.5 Real-world Datasets

We benchmark SIVF using three standard datasets representing diverse modalities: SIFT1M [22] (128D, vision), T2I-1B [39] (200D, multimodal/RAG), and GIST1M [22] (960D, high-dim features). While the total dataset size (1M vectors) is VRAM-bound, these experiments validate performance within the *active window* of streaming applications. Our Sliding Window benchmark (Sec. 4.7) further tests stability under high-churn workloads.

High-Throughput Ingestion. As shown in Figure 5, SIVF dominates ingestion scalability. On SIFT1M, it achieves 3.78M vec/s (105× speedup). On T2I-1B, typical of RAG workloads, SIVF sustains 2.91M vec/s (84× speedup vs. 34.6K vec/s). Even on GIST1M, SIVF maintains 850K vec/s (36× speedup), confirming that our pre-allocated slab architecture effectively saturates GPU bandwidth by eliminating dynamic resizing overhead.

Low-Latency Deletion. Figure 6 highlights the critical performance divergence. Baseline latency scales poorly due to CPU-GPU roundtrips, spiking to 11.8s for GIST1M. In contrast, SIVF's in-place bitmap updates decouple latency from dimension, maintaining sub-millisecond performance (≈0.9 ms) across all datasets. On T2I-1B, this yields a 2,770× improvement (2.4s → 0.87 ms), validating SIVF's feasibility for real-time streams.

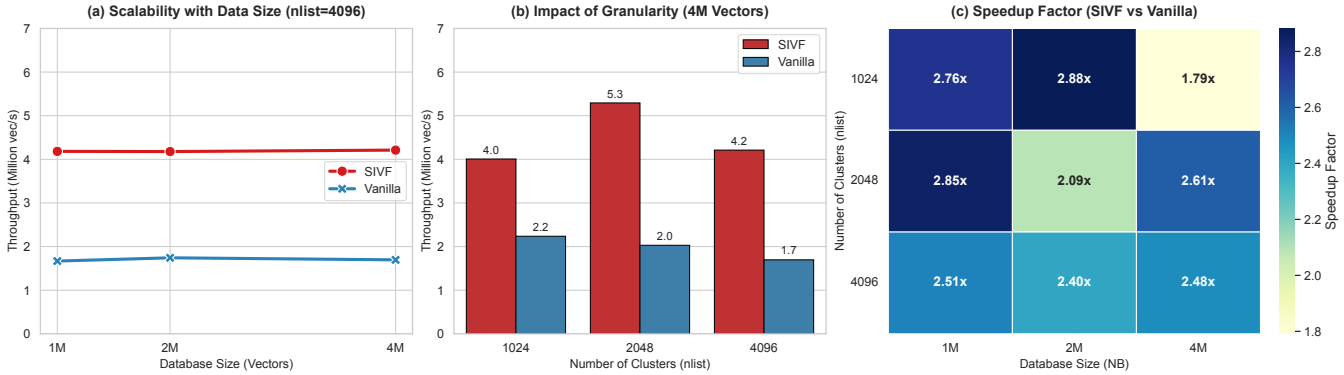

Figure 2: Performance evaluation of vector ingestion.

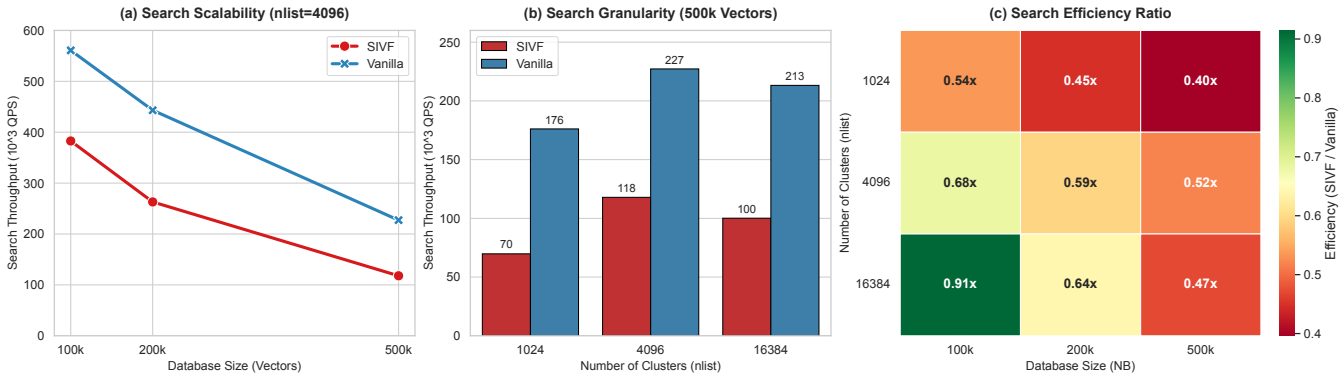

Figure 3: Performance evaluation of vector search.

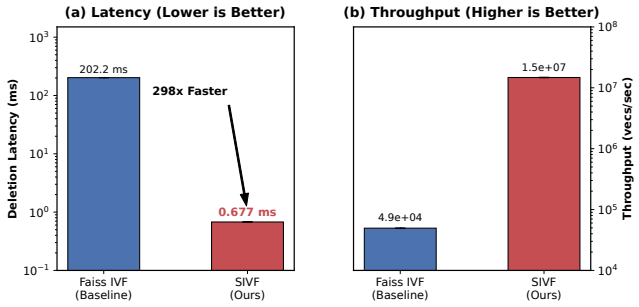

Figure 4: Performance comparison of vector deletion between Faiss IVF (Baseline) and SIVF.

Search Performance and Trade-offs. Figure 7 compares query throughput (QPS). SIVF outperforms the baseline on SIFT1M (1.5×) and retains 95.5% of performance on T2I-1B (17.8K vs 18.6K QPS), proving negligible overhead for moderate dimensions. On high-dimensional GIST1M, SIVF achieves 37% of baseline QPS. This trade-off results from reduced memory coalescing efficiency when traversing non-contiguous slabs with large vectors—a necessary compromise to enable instant, lock-free mutability.

4.6 Parameter Sensitivity

We evaluate system robustness by sweeping three parameters: (i) vector capacity factor (*maxvec_factor*), (ii) slab pool redundancy (*slab_factor*), and (iii) deletion batch size. These control logical headroom, pre-allocated memory for bursts, and the trade-off between amortization and freshness. Figures 8 and 9 summarize the results.

Impact of Slab Management. As shown in Figure 8, SIVF maintains consistent insertion throughput (2.7M–3.2M vec/s) across configurations. Increasing *slab_factor* from 1.0 to 1.3 improves performance by expanding the pool of free slabs, which reduces allocation contention during bursts. Conversely, performance is insensitive to *maxvec_factor*, confirming that the slab abstraction effectively decouples logical capacity from physical memory pressure. Insertion costs remain bounded by $O(1)$ operations (payload write, bitmap update, address mapping) provided the slab pool is sufficiently provisioned.

Effect of Deletion Batch Size. Deletion performance is driven by amortization. Larger batches improve throughput by amortizing fixed kernel launch and lookup overheads, while smaller batches minimize staleness but expose these costs. This behavior aligns with SIVF's $O(1)$ logical deletion mechanism, where work is limited to an address lookup and atomic bit flip.

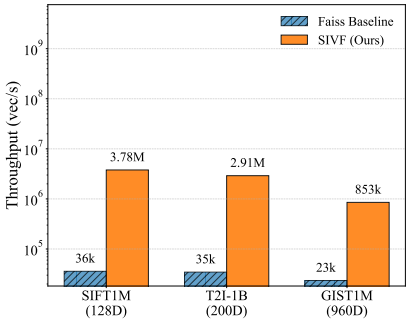

Figure 5: Ingestion throughput.

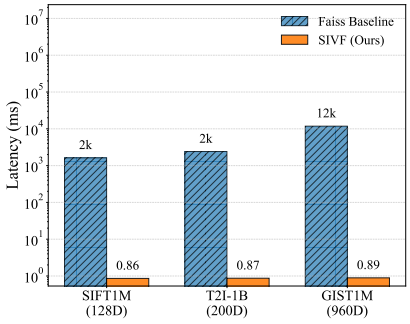

Figure 6: Deletion latency.

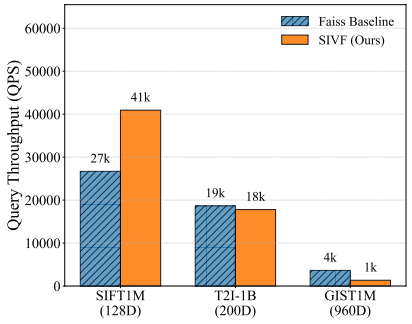

Figure 7: Search performance (QPS).

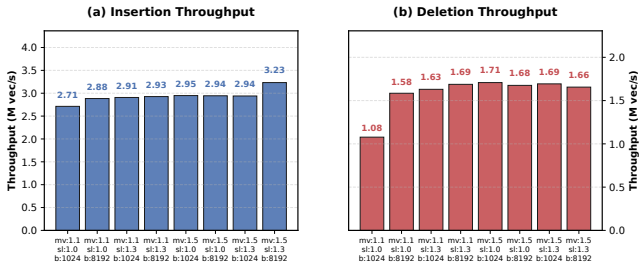

Figure 8: Throughput sensitivity analysis.

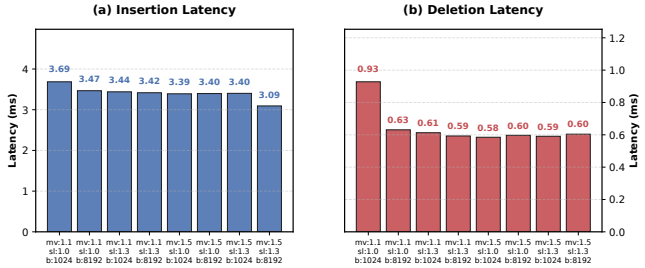

Figure 9: Latency sensitivity analysis.

Stable Sub-Millisecond Deletion. Crucially, Figure 9 shows that deletion latency remains strictly sub-millisecond (0.58–0.93 ms) across all settings. Because deletions require no structural reorganization (e.g., compaction or CPU–GPU roundtrips), performance is independent of list occupancy. Even with 1,000-vector batches, SIVF sustains > 1.7M vec/s throughput, validating its stability for high-churn workloads.

4.7 End-to-End Streaming Performance

We evaluate real-time applicability using a *Sliding Window* benchmark that mimics production streams. The system maintains a fixed active window (W) by ingesting a new batch (B) and evicting the oldest batch (B) at each step. We test on SIFT1M ($W = 200$K, $B = 10$K) and GIST1M ($W = 100$K, $B = 5$K).

As shown in Figure 10, the standard Faiss baseline suffers from high latency due to the CPU-GPU roundtrip required for index reconstruction, causing system freezes of 355 ms (SIFT1M) and >1.1 s (GIST1M) per update. In contrast, SIVF executes updates strictly in VRAM via slab-based management, reducing per-step latency to ≈2.2 ms (SIFT1M) and 4.2 ms (GIST1M). This yields speedups of 161× and 266×, respectively. Notably, the performance gap widens with dimensionality (GIST1M), where the baseline is bottlenecked by PCIe saturation. SIVF eliminates this bottleneck, maintaining single-digit millisecond latency and transforming the GPU IVF index into a real-time streaming engine.

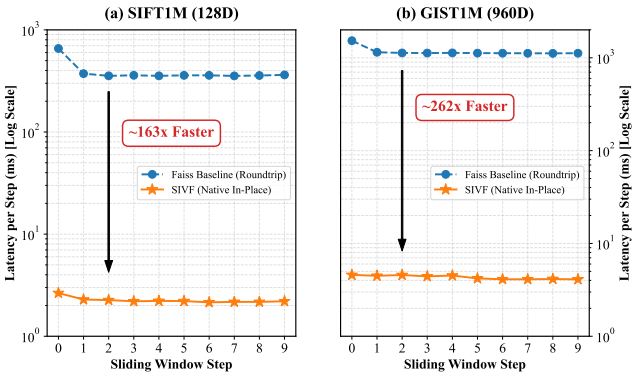

Figure 10: End-to-End Streaming Performance.

4.8 Memory Efficiency and Scalability

A common concern with dynamic graph-based or list-based indices is the potential for memory bloating due to structural overhead, such as pointers and metadata, as well as fragmentation. To quantify the space complexity of SIVF, we conducted a memory footprint analysis comparing the allocated VRAM usage of SIVF against a theoretically compact array baseline across varying dataset sizes ranging from 100K to 1M vectors.

Figure 11 illustrates the memory growth trends for both SIFT1M and GIST1M. The results demonstrate that SIVF exhibits deterministic linear scalability with negligible structural overhead. For SIFT1M ($d = 128$), the overhead stabilizes at 0.77%, while for the

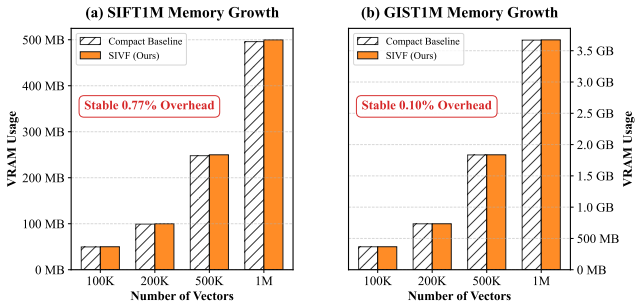

Figure 11: Memory Efficiency and Scalability.

Table 1: Comparison of ingestion throughput (K vec/s) and deletion latency (ms) across different datasets.

Method	SIFT1M		T2I-1B		GIST1M	
	Add	Del	Add	Del	Add	Del
Flat (no index)	9,169	836	6,020	1,160	1,230	1,143
HNSW [32]	25.5	N/A	25.7	N/A	0.6	N/A
NSG [8]	3.7	N/A	3.2	N/A	0.7	N/A
LSH [4]	787	14.6	428	16.3	36.6	9.2
SIVF (this work)	**3,780**	**0.86**	**2,910**	**0.87**	**853**	**0.89**

high-dimensional GIST1M ($d = 960$), it is merely 0.10%. This efficiency stems from our coarse-grained slab design, where the metadata cost (128-byte header) is amortized over a batch of 32 vectors. Consequently, SIVF achieves orders-of-magnitude performance gains in dynamic operations without imposing significant storage penalties compared to static, compact indices.

Furthermore, we verified the efficacy of our memory reclamation strategy. In a stress test involving the deletion of 50% of the dataset followed by immediate re-insertion, SIVF maintained a constant memory footprint without triggering OS-level deallocation. The deletion operation completed in sub-millisecond time per batch (e.g., 4.04 ms for 100K GIST vectors), confirming that memory slots are logically reclaimed via bitmap toggling and immediately available for reuse. This "zero-cost" reclamation ensures that SIVF can sustain long-running streaming workloads without suffering from memory leaks or fragmentation-induced bloat.

4.9 SIVF vs. Non-IVF Streaming

To position SIVF within the broader indexing landscape, we benchmarked it against four representative non-IVF architectures: (1) Faiss GPU Flat (brute-force baseline, i.e., no index), (2) HNSW [32] (dynamic graphs), (3) LSH [4] (legacy hash-based approaches), and (4) NSG [8] (static graphs).

Table 1 reports the results of both vector ingestion and deletion in a streaming context on three datasets. graph indices (HNSW, NSG) suffer catastrophic performance degradation on high-dimensional data, dropping to merely ~600 vec/s on GIST1M due to cache thrashing, and fail to support native deletion (N/A). While GPU Flat achieve the highest ingestion throughput because they perform brute-force search without indexing overhead, they incur prohibitive deletion latencies (>1s) due to CPU-GPU roundtrips. SIVF emerges as the only architecture enabling holistic stream processing: it sustains GPU-class ingestion rates (e.g., 853K vec/s on GIST1M, orders of magnitudes faster than graphs) while providing sub-millisecond native deletion, effectively bridging the gap between static high-throughput search and dynamic mutability.

5 Conclusion

In this work, we addressed the structural limitation inherent in standard GPU-accelerated Inverted File (IVF) indices, which effectively precludes their use in streaming vector search due to rigid and static memory layouts. We proposed SIVF, a new GPU-native architecture that relocates memory management responsibilities entirely onto the device. By synthesizing a slab-based memory allocator, a lock-free validity bitmap, and a GPU-resident address translation table, SIVF decouples index maintenance from the host CPU to enable high-throughput and in-place mutation directly in VRAM. Our comprehensive evaluation against the industry-standard Faiss library, spanning five baseline architectures (GPU Flat, GPU IVF, HNSW, NSG, and LSH) and three real-world datasets (SIFT, GIST, T2I), demonstrates that SIVF maintains robust performance. Specifically, SIVF accelerates data deletion by up to $13,300\times$ to achieve sub-millisecond latency and improves ingestion throughput by orders of magnitude compared to those baselines. In end-to-end sliding window scenarios, SIVF eliminates system freezes and achieves a $161\times$ to $266\times$ speedup while maintaining negligible memory overhead ($< 0.8\%$) compared to compact static indices. We believe SIVF represents a significant step towards enabling real-time, mutable vector search on GPU platforms, unlocking new possibilities for applications based on streaming vectors.

Despite the orders-of-magnitude improvement in mutation latency, our current SIVF design entails specific trade-offs.

First, the *pointer-chasing overhead* inherent in the linked-slab architecture reduces memory coalescing efficiency for those high-dimensional vectors. As observed in the GIST1M ($d = 960$) evaluation, this results in a $2.7\times$ reduction in raw search QPS compared to contiguous static indices. While acceptable for write-heavy streaming scenarios, future work will address this by exploring *adaptive super-slabs* that enforce page-aligned storage to restore memory bandwidth utilization.

Second, the current implementation operates on a single GPU, bounding the maximum active window size to the available VRAM (e.g., approx. 20M 128-dim vectors on a 24 GB card). While our sliding window experiments demonstrate stability under infinite streams, scaling the *simultaneous* index capacity to billion-scale datasets will require extending SIVF with *multi-GPU sharding* via NVLink and Unified Memory prefetching techniques.

Finally, our consistency model prioritizes throughput over strict linearizability. While sufficient for the probabilistic nature of ANN search, future work will investigate lightweight snapshot isolation mechanisms to support transactional semantics without compromising the lock-free ingestion speed.

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Chatzakis, M., Papakonstantinou, Y., and Palpanas, T. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 115–128.

[3] Chung, J. W., Lin, H., and Zhao, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.

[4] Datar, M., Immorlica, N., Indyk, P., and Mirrokni, V. S. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (New York, NY, USA, 2004), SCG '04, Association for Computing Machinery, p. 253–262.

[5] Deng, L., Chen, P., Zeng, X., Wang, T., Zhao, Y., and Zheng, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 3 (Nov. 2024), 812–821.

[6] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[7] Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).

[8] Fu, C., Xiang, C., Wang, C., and Cai, D. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow. 12*, 5 (Jan. 2019), 461–474.

[9] Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., and Wong, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[10] Gao, J., and Long, C. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data 2*, 3 (May 2024).

[11] Gong, Z., Zeng, Y., and Chen, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow. 18*, 10 (June 2025), 3518–3530.

[12] Gou, Y., Gao, J., Xu, Y., and Long, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[13] Hua, Z., Mo, Q., Yao, Z., Cui, L., Liu, X., Wang, G., Wei, Z., Liu, X., Tang, T., Liu, S., and Qu, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[14] Hui, X., Xu, Y., Guo, Z., and Shen, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 42–55.

[15] Hui, X., Xu, Y., and Shen, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[16] Hwang, S., Lee, E., Oh, H., and Yi, Y. Fasop: Fast yet accurate automated search for optimal parallelization of transformers on heterogeneous gpu clusters. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 253–266.

[17] Indyk, P., and Xu, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[18] Jääsaari, E., Hyvönen, V., and Roos, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[19] Jiang, M., Yang, Z., Zhang, F., Hou, G., Shi, J., Zhou, W., Li, F., and Wang, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[20] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[21] Jung, S., Park, Y., Lee, H., Oh, Y. H., and Lee, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.

[22] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[23] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[24] Laut, S., Borrell, R., and Casas, M. Extending sparse patterns to improve inverse preconditioning on gpu architectures. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 200–213.

[25] Leyba, K., Hofmeyr, S., Forrest, S., Cannon, J., and Moses, M. Simcov-gpu: Accelerating an agent-based model for exascale. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 322–333.

[26] Li, M., Yan, X., Lu, B., Zhang, Y., Cheng, J., and Ma, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[27] Li, X., Laguna, I., Fang, B., Swirydowicz, K., Li, A., and Gopalakrishnan, G. Design and evaluation of gpu-fpx: A low-overhead tool for floating-point exception detection in nvidia gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 59–71.

[28] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[29] Liang, A., Zhang, P., Yao, B., Chen, Z., Song, Y., and Cheng, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow. 18*, 4 (Dec. 2024), 1118–1130.

[30] Liu, M., Zhong, S., Yang, Q., Han, Y., Liu, X., and Ma, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.

[31] Lu, K., Xiao, C., and Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).

[32] Malkov, Y. A., and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. vol. 42, IEEE Computer Society, p. 824–836.

[33] Maurya, A., Rafique, M. M., Tonellot, T., AlSalem, H. J., Cappello, F., and Nicolae, B. Gpu-enabled asynchronous multi-level checkpoint caching and prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 73–85.

[34] Miao, D., Laguna, I., and Rubio-González, C. Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[35] Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., and Wang, Y. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (Los Alamitos, CA, USA, May 2024), IEEE Computer Society, pp. 4236–4247.

[36] Peng, Z., Qiao, M., Zhou, W., Li, F., and Deng, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 10 (June 2025), 3256–3268.

[37] Peng, Z., Thomadakis, P., Pienaar, J., and Kestor, G. Liteform: Lightweight and automatic format composition for sparse matrix-matrix multiplication on gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[38] Qiu, R., and Tang, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[39] simhadri, H. V., Aumüller, M., Douze, M., Baranchuk, D., Ingber, A., Liberty, E., Williams, G., Landrum, B., Manohar, M. D., Karjikar, M., Dhulipala, L., Chen, M., Chen, Y., Ma, R., Zhang, K., Cai, Y., Shi, J., Zheng, W., Chen, Y., Yin, J., and Huang, B. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2025).

[40] Sun, P., Simcha, D., Dopson, D., Guo, R., and Kumar, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information*

Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023 (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[41] Tran, A., Laguna, I., and Gopalakrishnan, G. Fpboxer: Efficient input-generation for targeting floating-point exceptions in gpu programs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 83–93.

[42] Voruganti, S., and Özsu, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[43] Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[44] Wang, Z., Zhang, J., and Hu, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[45] Wei, J., Lee, X., Liao, Z., Palpanas, T., and Peng, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[46] Xu, Q., Zhang, F., Li, C., Cao, L., Chen, Z., Zhai, J., and Du, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[47] Yang, M., Cai, Y., and Zheng, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[48] Zhang, B., Tian, J., Di, S., Yu, X., Feng, Y., Liang, X., Tao, D., and Cappello, F. Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 129–142.

[49] Zhang, F., Jiang, M., Hou, G., Shi, J., Fan, H., Zhou, W., Li, F., and Wang, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[50] Zhong, X., Li, H., Jin, J., Yang, M., Chu, D., Wang, X., Shen, Z., Jia, W., Gu, G., Xie, Y., Lin, X., Shen, H. T., Song, J., and Cheng, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 12 (Aug. 2025), 5017–5030.